

Syllabus (Unit 1)

1.1 Introduction to PHP

- 1.1.1 Understanding the role of PHP in server-side web development
- 1.1.2 History and evolution of PHP
- 1.1.3 Installation and configuration using XAMPP/WAMP
- 1.1.4 Setting up the development environment using Visual Studio Code

1.2 Basic PHP Syntax and Variables

- 1.2.1 PHP script structure and tags
- 1.2.2 Declaring and using variables and constants
- 1.2.3 Using echo and print statements
- 1.2.4 Comments and formatting conventions

1.3 Data Types and Operators

- 1.3.1 Primitive types: string, int, float, boolean
- 1.3.2 Arrays and objects
- 1.3.3 Type casting and type juggling
- 1.3.4 Operators: arithmetic, logical, comparison, assignment

1.4 Control Structures and Arrays

- 1.4.1 Conditional statements: if, else, elseif, switch
- 1.4.2 Looping constructs: for, while, do-while, foreach
- 1.4.3 Arrays: indexed, associative, multidimensional
- 1.4.4 Array operations: sort(), asort(), ksort(), array_merge()

1.5 Functions and Form Handling

- 1.5.1 Creating and invoking user-defined functions
- 1.5.2 Function parameters and return values
- 1.5.3 Variable scope: global vs. local
- 1.5.4 Handling forms with \$_GET and \$_POST
- 1.5.5 Basic input validation and sanitization

Concepts of Php and introduction

PHP started out as a small open source project that evolved as more and more people found out how useful it was. Rasmus Lerdorf unleashed the first version of PHP way back in 1994.

- PHP is a recursive acronym for "PHP: Hypertext Preprocessor".
- PHP is a server side scripting language that is embedded in HTML. It is used to manage dynamic content, databases, session tracking, even build entire e-commerce sites.
- It is integrated with a number of popular databases, including MySQL, PostgreSQL, Oracle, Sybase, Informix, and Microsoft SQL Server.
- PHP is pleasingly zippy in its execution, especially when compiled as an Apache module on the Unix side. The MySQL server, once started, executes even very complex queries with huge result sets in record-setting time.
- PHP Syntax is C-Like.

Common uses of PHP

- PHP performs system functions, i.e. from files on a system it can create, open, read, write, and close them.
- PHP can handle forms, i.e. gather data from files, save data to a file, through email you can send data, return data to the user.
- You add, delete, and modify elements within your database through PHP.
- Access cookies variables and set cookies.
- Using PHP, you can restrict users to access some pages of your website.
- It can encrypt data.

Characteristics of PHP

Five important characteristics make PHP's practical nature possible –

- Simplicity
- Efficiency
- Security
- Flexibility
- Familiarity

"Hello World" Script in PHP

To get a feel for PHP, first start with simple PHP scripts. Since "Hello, World!" is an essential example, first we will create a friendly little "Hello, World!" script.

Unit 1: Core PHP Programming

As mentioned earlier, PHP is embedded in HTML. That means that in amongst your normal HTML (or XHTML if you're cutting-edge) you'll have PHP statements like this –

```
<html>
```

```
<head>
```

```
<title>Hello World</title>
```

```
</head>
```

```
<body>
```

```
<?php echo "Hello, World!";?>
```

```
</body>
```

```
</html>
```

Output:

Hello, World!

What is PHP?

- **PHP (Hypertext Preprocessor)** is a widely-used, open-source server-side scripting language.
- It is embedded within HTML to manage dynamic content, session tracking, databases, and more.

Role in Web Development:

- **Server-Side Processing:** PHP code is executed on the server, and only the plain HTML result is sent to the client browser.
- **Database Interaction:** Works efficiently with databases like MySQL to retrieve, store, and manipulate data.
- **Form Handling:** Processes data submitted by users through web forms.
- **Session and Cookie Management:** Maintains user sessions, logins, and preferences.
- **Security:** Handles authentication, data encryption, and input validation.
- **Content Management Systems (CMS):** Backbone of popular CMS like WordPress, Joomla, and Drupal.
- **APIs and Web Services:** Can consume and build RESTful APIs.

Unit 1: Core PHP Programming

1.1.2 History and Evolution of PHP

Timeline:

- **1994:** Created by Rasmus Lerdorf as a set of Common Gateway Interface (CGI) binaries written in C to track visits to his resume online.
- **1995:** Released as **Personal Home Page Tools (PHP Tools)**.
- **1997:** PHP 3 released - major rewrite, supported databases, became widely popular.
- **2000:** PHP 4 introduced - improved performance, session handling.
- **2004:** PHP 5 released - introduced Object-Oriented Programming (OOP), PDO, better XML handling.
- **2015:** PHP 7 released - significant performance improvements, type declarations, error handling enhancements.
- **2020:** PHP 8 introduced - Just-in-Time (JIT) compilation, union types, improved performance.

Key Points:

- PHP evolved from simple scripts to a full-fledged, powerful programming language.
 - Continuously improved to support modern web development needs like OOP, security, and faster processing.
-

1.1.3 Installation and Configuration using XAMPP/WAMP

What is XAMPP?

- A free, open-source cross-platform web server solution package.
- Includes **Apache**, **MySQL/MariaDB**, **PHP**, and **Perl**.

What is WAMP?

- A Windows-based web development environment.
- Includes **Windows + Apache + MySQL + PHP**.

Installation Steps (Common for XAMPP/WAMP):

1. **Download** the XAMPP/WAMP installer from the official website.
2. **Run the Installer** and follow the setup wizard.
3. Select components (Apache, MySQL, PHP) to install.
4. Choose the installation directory (default is C:\xampp or C:\wamp).
5. Complete the installation.

Configuration:

- Start the **Apache** and **MySQL** services from the XAMPP/WAMP control panel.
 - Access the localhost via browser:
-

Unit 1: Core PHP Programming

- **XAMPP:** http://localhost/dashboard/
 - **WAMP:** http://localhost/
- Place PHP files inside:
 - **XAMPP:** C:\xampp\htdocs
 - **WAMP:** C:\wamp\www
- Test PHP Setup:

```
<?php
phpinfo();
?>
```

Save as test.php in htdocs or www folder and run: http://localhost/test.php

Variables

variable starts with the \$ sign, followed by the name of the variable:

```
<?php
$txt = "Hello world!";
$x = 5;
$y = 10.5;
?>
```

Rules for PHP variables:

- A variable starts with the \$ sign, followed by the name of the variable
- A variable name must start with a letter or the underscore character
- A variable name cannot start with a number
- A variable name can only contain alpha-numeric characters and underscores (A-z, 0-9, and _)
- Variable names are case-sensitive (\$age and \$AGE are two different variables)

Note: Remember that PHP variable names are case-sensitive!

Variable Scope

PHP has three types of variable scopes:

1. Global variable
2. Local variable
3. Static variable

Global

A variable declared **outside** a function has a GLOBAL SCOPE and can only be accessed outside a function:

Example

Unit 1: Core PHP Programming

Variable with global scope:

```
<?php
$name = "Sanaya Sharma";    //Global Variable
function global_var()
{
    global $name;
    echo "Variable inside the function: ". $name;
    echo "</br>";
}
global_var();
echo "Variable outside the function: ". $name;
?>
```

Another way to use the global variable inside the function is predefined \$GLOBALS array.

```
<?php
$num1 = 5;    //global variable
$num2 = 13;    //global variable
function global_var()
{
    $sum = $GLOBALS['num1'] + $GLOBALS['num2'];
    echo "Sum of global variables is: " . $sum;
}
global_var();
?>
```

Local

A variable declared **within** a function has a LOCAL SCOPE and can only be accessed within that function:

Example

Variable with local scope:

```
<?php
function myTest() {
    $x = 5; // local scope
    echo "<p>Variable x inside function is: $x</p>";
}
myTest();
```

// using x outside the function will generate an error

Unit 1: Core PHP Programming

```
echo "<p>Variable x outside function is: $x</p>";  
?>
```

Static

It is a feature of PHP to delete the variable, once it completes its execution and memory is freed. Sometimes we need to store a variable even after completion of function execution. Therefore, another important feature of variable scoping is static variable. We use the static keyword before the variable to define a variable, and this variable is called as **static variable**. Static variables exist only in a local function, but it does not free its memory after the program execution leaves the scope. Understand it with the help of an example:

Example:

```
<?php  
    function static_var()  
    {  
        static $num1 = 3;    //static variable  
        $num2 = 6;          //Non-static variable  
        //increment in non-static variable  
        $num1++;  
        //increment in static variable  
        $num2++;  
        echo "Static: " . $num1 . "<br>";  
        echo "Non-static: " . $num2 . "<br>";  
    }  
  
    //first function call  
    static_var();  
  
    //second function call  
    static_var();  
?>
```

Output

Static: 4

Non-static: 7

Static: 5

Non-static: 7

Unit 1: Core PHP Programming

PHP \$ and \$\$ Variables

The **\$var** (single dollar) is a normal variable with the name var that stores any value like string, integer, float, etc.

The **\$\$var** (double dollar) is a reference variable that stores the value of the \$variable inside it. To understand the difference better, let's see some examples.

Example

```
<?php
$x = "abc";
$$x = 200;
echo $x."<br/>";
echo $$x."<br/>";
echo $abc;
?>
```

Output

```
abc
200
200
```

Constants

PHP constants are name or identifier that can't be changed during the execution of the script except .

1. Using define() function
2. Using const keyword

Syntax

define(name, value)

Example

```
<?php
define("MSG1","Hello JavaTpoint PHP");
echo MSG1;
const MSG2="Hello const by JavaTpoint PHP";
echo MSG2;
?>
```


Unit 1: Core PHP Programming

echo

The echo statement can be used with or without parentheses: echo or echo().

Display Text

The following example shows how to output text with the echo command

Example:

```
<?php
echo "<h2>PHP is Fun!</h2>";
echo "Hello world!<br>";
$txt1 = "Learn PHP";
$txt2 = "W3Schools.com";
$x = 5;
$y = 4;

echo "<h2>" . $txt1 . "</h2>";
echo "Study PHP at " . $txt2 . "<br>";
echo $x + $y;
echo "Hello by PHP echo
this is multi line
text printed by
PHP echo statement
";
echo "Hello escape \"sequence\" characters";

?>
```

Print

The print statement can be used with or without parentheses: print or print().

Display Text

The following example shows how to output text with the print command (notice that the text can contain HTML markup):

Example

```
<?php
print "<h2>PHP is Fun!</h2>";
print "Hello world!<br>";
print ("Hello by PHP print()");
?>
```

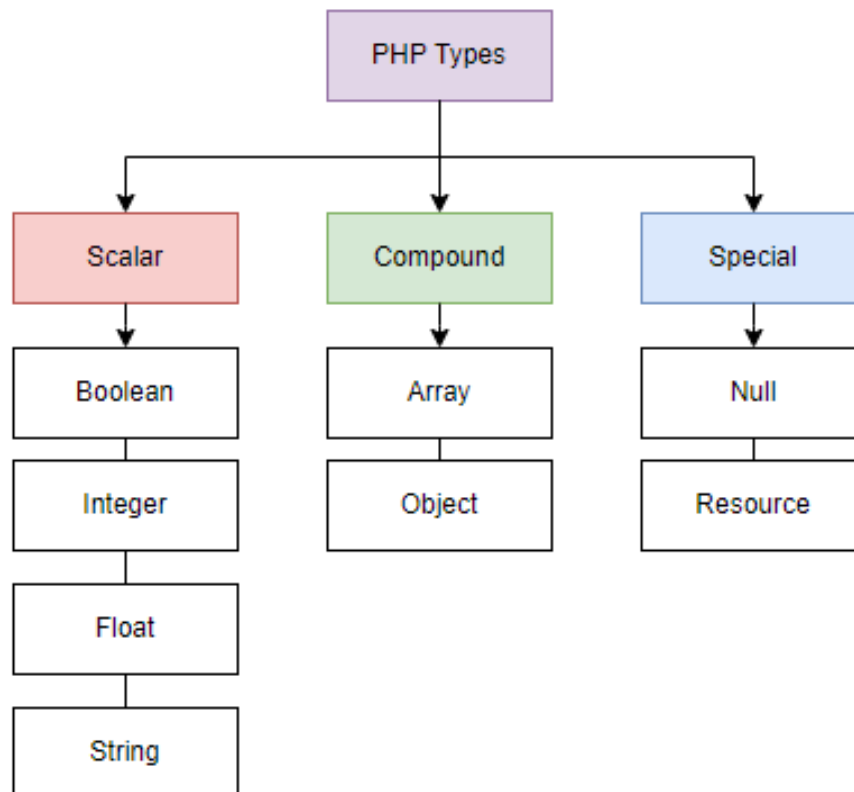
Unit 1: Core PHP Programming

Echo VS Print

Echo	Print
1. echo does not return any value. 2. We can pass multiple strings separated by comma (,) in echo. 3. echo is faster than print statement.	1. print always returns an integer value, which is 1. 2. Using print, we cannot pass multiple arguments. 3. print is slower than echo statement.

Data types

A type specifies the amount of memory that allocates to a value associated with it.



Scalar type:

It hold single value only:

1. Boolean: Booleans are the simplest data type works like switch. It holds only two values: **TRUE (1)** or **FALSE (0)**
2. Integer: Integer means numeric data with a negative or positive sign. It holds only whole numbers, i.e., numbers without fractional part or decimal points. The

Unit 1: Core PHP Programming

range of an integer must lie between 2,147,483,648 and 2,147,483,647 i.e., - 2^{31} to 2^{31} .

3. Float: A floating-point number is a number with a decimal point.
4. String: A string is a non-numeric data type. It holds letters or any alphabets, numbers, and even special characters. String values must be enclosed either within single quotes or in double quotes. But both are treated differently.

Example: `$name = "Raman";`

`//both single and double quote statements will treat different`

`echo "Hello $name";`

`echo "</br>";`

`echo 'Hello $name';`

Output:

Hello Javatpoint

Hello \$company

Compound Type:

It hold multiple values

1. Array: An array is a compound data type. It can store multiple values of same data type in a single variable.

Example:

`$scores = [1, 2, 3];`

2. Objects are the instances of user-defined classes that can store both values and functions.

Special type

1. Resource: Resources are not the exact data type in PHP. Basically, these are used to store some function calls or references to external PHP resources. For example - a database call.
2. Null: Null is a special data type that has only one value: NULL

Operators

PHP divides the operators in the following groups:

- Arithmetic operators: The PHP arithmetic operators are used to perform common arithmetic operations such as addition, subtraction, etc. with numeric values.

Operator	Name	Example	Explanation
+	Addition	<code>\$a + \$b</code>	Sum of operands
-	Subtraction	<code>\$a - \$b</code>	Difference of operands

Unit 1: Core PHP Programming

*	Multiplication	$\$a * \b	Product of operands
/	Division	$\$a / \b	Quotient of operands
%	Modulus	$\$a \% \b	Remainder of operands
**	Exponentiation	$\$a ** \b	$\$a$ raised to the power $\$b$

- Assignment operators: The assignment operators are used to assign value to different variables. The basic assignment operator is "=".

Operator	Name	Example	Explanation
=	Assign	$\$a = \b	The value of right operand is assigned to the left operand.
+=	Add then Assign	$\$a += \b	Addition same as $\$a = \$a + \$b$
-=	Subtract then Assign	$\$a -= \b	Subtraction same as $\$a = \$a - \$b$
*=	Multiply then Assign	$\$a *= \b	Multiplication same as $\$a = \$a * \$b$
/=	Divide then Assign (quotient)	$\$a /= \b	Find quotient same as $\$a = \$a / \$b$
%=	Divide then Assign (remainder)	$\$a \% = \b	Find remainder same as $\$a = \$a \% \$b$

- Bitwise operators

The bitwise operators are used to perform bit-level operations on operands. These operators allow the evaluation and manipulation of specific bits within the integer.

Operator	Name	Example	Explanation
&	And	$\$a \& \b	Bits that are 1 in both $\$a$ and $\$b$ are set to 1, otherwise 0.
	Or (Inclusive or)	$\$a \b	Bits that are 1 in either $\$a$ or $\$b$ are set to 1
^	Xor (Exclusive or)	$\$a \wedge \b	Bits that are 1 in either $\$a$ or $\$b$ are set to 0.

Unit 1: Core PHP Programming

~	Not	~\$a	Bits that are 1 set to 0 and bits that are 0 are set to 1
<<	Shift left	\$a << \$b	Left shift the bits of operand \$a \$b steps
>>	Shift right	\$a >> \$b	Right shift the bits of \$a operand by \$b number of places

- Comparison Operators

Comparison operators allow comparing two values, such as number or string. Below the list of comparison operators are given:

Operator	Name	Example	Explanation
==	Equal	\$a == \$b	Return TRUE if \$a is equal to \$b
===	Identical	\$a === \$b	Return TRUE if \$a is equal to \$b, and they are of same data type
!==	Not identical	\$a !== \$b	Return TRUE if \$a is not equal to \$b, and they are not of same data type
!=	Not equal	\$a != \$b	Return TRUE if \$a is not equal to \$b
<>	Not equal	\$a <> \$b	Return TRUE if \$a is not equal to \$b
<	Less than	\$a < \$b	Return TRUE if \$a is less than \$b
>	Greater than	\$a > \$b	Return TRUE if \$a is greater than \$b
<=	Less than or equal to	\$a <= \$b	Return TRUE if \$a is less than or equal \$b
>=	Greater than or equal to	\$a >= \$b	Return TRUE if \$a is greater than or equal \$b
<=>	Spaceship	\$a <=> \$b	Return -1 if \$a is less than \$b Return 0 if \$a is equal \$b Return 1 if \$a is greater than \$b

- Incrementing/Decrementing Operators

The increment and decrement operators are used to increase and decrease the value of a variable.

Unit 1: Core PHP Programming

Operator	Name	Example	Explanation
++	Increment	++\$a	Increment the value of \$a by one, then return \$a
		\$a++	Return \$a, then increment the value of \$a by one
--	decrement	--\$a	Decrement the value of \$a by one, then return \$a
		\$a--	Return \$a, then decrement the value of \$a by one

- Logical Operators

The logical operators are used to perform bit-level operations on operands. These operators allow the evaluation and manipulation of specific bits within the integer.

Operator	Name	Example	Explanation
And	And	\$a and \$b	Return TRUE if both \$a and \$b are true
Or	Or	\$a or \$b	Return TRUE if either \$a or \$b is true
Xor	Xor	\$a xor \$b	Return TRUE if either \$ or \$b is true but not both
!	Not	! \$a	Return TRUE if \$a is not true
&&	And	\$a && \$b	Return TRUE if either \$a and \$b are true
	Or	\$a \$b	Return TRUE if either \$a or \$b is true

- String Operators

The string operators are used to perform the operation on strings. There are two string operators in PHP, which are given below:

Operator	Name	Example	Explanation
.	Concatenation	\$a . \$b	Concatenate both \$a and \$b
.=	Concatenation and Assignment	\$a .= \$b	First concatenate \$a and \$b, then assign the concatenated string to \$a, e.g. \$a = \$a . \$b

- Array Operators

The array operators are used in case of array. Basically, these operators are used to compare the values of arrays.

Unit 1: Core PHP Programming

Operator	Name	Example	Explanation
+	Union	<code>\$a + \$y</code>	Union of <code>\$a</code> and <code>\$b</code>
<code>==</code>	Equality	<code>\$a == \$b</code>	Return TRUE if <code>\$a</code> and <code>\$b</code> have same key/value pair
<code>!=</code>	Inequality	<code>\$a != \$b</code>	Return TRUE if <code>\$a</code> is not equal to <code>\$b</code>
<code>===</code>	Identity	<code>\$a === \$b</code>	Return TRUE if <code>\$a</code> and <code>\$b</code> have same key/value pair of same type in same order
<code>!==</code>	Non-Identity	<code>\$a !== \$b</code>	Return TRUE if <code>\$a</code> is not identical to <code>\$b</code>
<code><></code>	Inequality	<code>\$a <> \$b</code>	Return TRUE if <code>\$a</code> is not equal to <code>\$b</code>

Conditional Statements

to write code that perform different actions based on the results of a logical or comparative test conditions at run time.

- The **if** statement
- The **if...else** statement
- The **if...elseif....else** statement
- The **switch...case** statement

The if Statement

The *if* statement is used to execute a block of code only if the specified condition evaluates to true.

Syntax:

```
if(condition){  
    // Code to be executed  
}
```

Example:

output "Have a nice weekend!" if the current day is Friday:

```
<?php  
$d = date("D");  
if($d == "Fri"){  
    echo "Have a nice weekend!";  
}
```

Unit 1: Core PHP Programming

?>

The if...else Statement

You can enhance the decision making process by providing an alternative choice through adding an *else* statement to the *if* statement. The *if...else* statement allows you to execute one block of code if the specified condition is evaluates to true and another block of code if it is evaluates to false. It can be written, like this:

Syntax:

```
if(condition){  
    // Code to be executed if condition is true  
} else{  
    // Code to be executed if condition is false  
}
```

Exaple:Output "Have a nice weekend!" if the current day is Friday, otherwise it will output "Have a nice day!"

```
<?php  
$d = date("D");  
if($d == "Fri"){  
    echo "Have a nice weekend!";  
} else{  
    echo "Have a nice day!";  
}  
?>
```

The if...elseif...else Statement

The *if...elseif...else* a special statement that is used to combine multiple *if...else* statements.

Syntax:

```
if(condition1){  
    // Code to be executed if condition1 is true  
} elseif(condition2){  
    // Code to be executed if the condition1 is false and condition2 is true  
} else{  
    // Code to be executed if both condition1 and condition2 are false  
}
```

Example:

output "Have a nice weekend!" if the current day is Friday, and "Have a nice Sunday!" if the current day is Sunday, otherwise it will output "Have a nice day!"

```
<?php
```


Unit 1: Core PHP Programming

```
$d = date("D");
if($d == "Fri"){
    echo "Have a nice weekend!";
} elseif($d == "Sun"){
    echo "Have a nice Sunday!";
} else{
    echo "Have a nice day!";
}
?>
```

The switch-case statement is an alternative to the if-elseif-else statement, which does almost the same thing. The switch-case statement tests a variable against a series of values until it finds a match, and then executes the block of code corresponding to that match.

```
switch(n){
    case label1:
        // Code to be executed if n=label1
        break;
    case label2:
        // Code to be executed if n=label2
        break;
    ...
    default:
        // Code to be executed if n is different from all labels
}
```

Consider the following example, which display a different message for each day.

Example

```
<?php
$today = date("D");
switch($today){
    case "Mon":
        echo "Today is Monday. Clean your house.";
        break;
    case "Tue":
        echo "Today is Tuesday. Buy some food.";
        break;
    case "Wed":
        echo "Today is Wednesday. Visit a doctor.";
```

Unit 1: Core PHP Programming

```
        break;
    case "Thu":
        echo "Today is Thursday. Repair your car.";
        break;
    case "Fri":
        echo "Today is Friday. Party tonight.";
        break;
    case "Sat":
        echo "Today is Saturday. Its movie time.";
        break;
    case "Sun":
        echo "Today is Sunday. Do some rest.";
        break;
    default:
        echo "No information available for that day.";
        break;
}
?>
```

Arrays

It is used to hold multiple values of similar type in a single variable.

Advantage of PHP Array

Less Code: We don't need to define multiple variables.

Easy to traverse: By the help of single loop, we can traverse all the elements of an array.

Sorting: We can sort the elements of array.

There are 3 types of array in PHP.

1. Indexed Array
2. Associative Array
3. Multidimensional Array

Indexed Array

PHP index is represented by number which starts from 0. We can store number, string and object in the PHP array. All PHP array elements are assigned to an index number by default.

There are two ways to define indexed array:

```
$season=array("summer","winter","spring","autumn");
echo "Season are: $season[0], $season[1], $season[2] and $season[3]";
```

OR

```
$season[0]="summer";
```

Unit 1: Core PHP Programming

```
$season[1]="winter";
$season[2]="spring";
$season[3]="autumn";
echo "Season are: $season[0], $season[1], $season[2] and $season[3]";
```

Associative Array

We can associate name with each array elements in PHP using => symbol.

There are two ways to define associative array:

```
$salary=array("Sonoo"=>"350000","John"=>"450000","Kartik"=>"200000");
echo "Sonoo salary: ".$salary["Sonoo"]."<br/>";
echo "John salary: ".$salary["John"]."<br/>";
echo "Kartik salary: ".$salary["Kartik"]."<br/>";
```

OR

```
$salary["Sonoo"]="350000";
$salary["John"]="450000";
$salary["Kartik"]="200000";
echo "Sonoo salary: ".$salary["Sonoo"]."<br/>";
echo "John salary: ".$salary["John"]."<br/>";
echo "Kartik salary: ".$salary["Kartik"]."<br/>";
```

Multidimensional Array

PHP multidimensional array is also known as array of arrays. It allows you to store tabular data in an array. PHP multidimensional array can be represented in the form of matrix which is represented by row * column.

Definition

```
$temp = array
(
    array(1,"sonoo",400000),
    array(2,"john",500000),
    array(3,"rahul",300000)
);
for ($row = 0; $row < 3; $row++) {
    for ($col = 0; $col < 3; $col++) {
        echo $temp[$row][$col]." ";
    }
    echo "<br/>";
}
```

Unit 1: Core PHP Programming

Php Loops

the following loop types:

- while - loops through a block of code as long as the specified condition is true
- do...while - loops through a block of code once, and then repeats the loop as long as the specified condition is true
- for - loops through a block of code a specified number of times
- foreach - loops through a block of code for each element in an array

while Loop

The while loop executes a block of code as long as the specified condition is true.

Syntax

```
while (condition is true) {  
    code to be executed;  
}
```

Examples

The example below displays the numbers from 1 to 5:

Example

```
<?php  
$x = 1;  
  
while($x <= 5) {  
    echo "The number is: $x <br>";  
    $x++;  
}  
?>
```

do...while Loop

The do...while loop will always execute the block of code once, it will then check the condition, and repeat the loop while the specified condition is true.

Syntax

```
do {  
    code to be executed;  
} while (condition is true);
```

Examples

The example below first sets a variable \$x to 1 (\$x = 1). Then, the do while loop will write some output, and then increment the variable \$x with 1. Then the condition is checked (is \$x less than, or equal to 5?), and the loop will continue to run as long as \$x is less than, or equal to 5:

Example

Unit 1: Core PHP Programming

```
<?php
$x = 1;

do {
    echo "The number is: $x <br>";
    $x++;
} while ($x <= 5);
?>
```

for Loop

The for loop is used when you know in advance how many times the script should run.

Syntax

```
for (init counter; test counter; increment counter) {
    code to be executed for each iteration;
}
```

Parameters:

- *init counter*: Initialize the loop counter value
- *test counter*: Evaluated for each loop iteration. If it evaluates to TRUE, the loop continues. If it evaluates to FALSE, the loop ends.
- *increment counter*: Increases the loop counter value

Examples

The example below displays the numbers from 0 to 10:

Example

```
<?php
for ($x = 0; $x <= 10; $x++) {
    echo "The number is: $x <br>";
}
?>
```

foreach Loop

The foreach loop works only on arrays, and is used to loop through each key/value pair in an array.

Syntax

```
foreach ($array as $value) {
    code to be executed;
}
```

For every loop iteration, the value of the current array element is assigned to \$value and the array pointer is moved by one, until it reaches the last array element.

Unit 1: Core PHP Programming

Examples

The following example will output the values of the given array (\$colors):

Example

```
<?php
$colors = array("red", "green", "blue", "yellow");

foreach ($colors as $value) {
    echo "$value <br>";
}
?>
```

Break

You have already seen the break statement used in an earlier chapter of this tutorial. It was used to "jump out" of a switch statement.

The break statement can also be used to jump out of a loop.

This example jumps out of the loop when **x** is equal to **4**:

Example

```
<?php
for ($x = 0; $x < 10; $x++) {
    if ($x == 4) {
        break;
    }
    echo "The number is: $x <br>";
}
?>
```

Continue

The continue statement breaks one iteration (in the loop), if a specified condition occurs, and continues with the next iteration in the loop.

This example skips the value of **4**:

Example

```
<?php
for ($x = 0; $x < 10; $x++) {
    if ($x == 4) {
        continue;
    }
    echo "The number is: $x <br>";
}
```

Unit 1: Core PHP Programming

```
}  
?>
```

Array Function:

Function	Description	Example	Output
array()	Creates an array	<pre>\$cars=array("Volvo","BMW","Toyota") \$age=array("Peter"=>"35","Ben"=>"37","Joe"=>"43"); \$cars=array (array("Volvo",100,96), array("BMW",60,59), array("Toyota",110,100))</pre>	
array_change_key_case()	Changes all keys in an array to lowercase or uppercase	<pre>\$age=array("Peter"=>"35","Ben"=>"37","Joe"=>"43"); print_r(array_change_key_case(\$age,CASE_LOWER));</pre>	Array ([peter] => 35 [ben] => 37 [joe] => 43)
array_chunk()	Splits an array into chunks of arrays	<pre>\$cars=array("Volvo","BMW","Toyota","Honda","Mercedes","Opel"); print_r(array_chunk(\$cars,2));</pre>	Array ([0] => Array ([0] => Volvo [1] => BMW) [1] => Array ([0] => Toyota [1] => Honda) [2] => Array ([0] => Mercedes [1] => Opel))
array_combine()	Creates an array by using the elements from one "keys" array and one "values" array	<pre>\$fname=array("Peter","Ben","Joe"); \$age=array("35","37","43"); \$c=array_combine(\$fname,\$age); print_r(\$c);</pre>	Array ([Peter] => 35 [Ben] => 37 [Joe] => 43)
array_count_values()	Counts all the values of an	<pre>\$a=array("A","Cat","Dog","A","Dog"); print_r(array_count_values(\$a));</pre>	Array ([A] => 2 [Cat] => 1 [Dog] => 2)

Unit 1: Core PHP Programming

	array		
array_diff()	Compare arrays, and returns the differences (compare values only)	<pre>\$a1=array("a"=>"red","b"=>"green", "b"=>"blue","d"=>"yellow"); \$a2=array("e"=>"red","f"=>"green", "b"=>"blue"); \$result=array_diff(\$a1,\$a2); print_r(\$result);</pre>	Array ([d] => yellow)
array_diff_assoc()	Compare arrays, and returns the differences (compare keys and values)	<pre>\$a1=array("a"=>"red","b"=>"green", "b"=>"blue","d"=>"yellow"); \$a2=array("a"=>"red","b"=>"green", "b"=>"blue"); \$result=array_diff_assoc(\$a1,\$a2); print_r(\$result);</pre>	Array ([d] => yellow)
array_diff_key()	Compare arrays, and returns the differences (compare keys only)	<pre>\$a1=array("a"=>"red","b"=>"green", "b"=>"blue"); \$a2=array("a"=>"red","c"=>"green", "d"=>"blue"); \$result=array_diff_key(\$a1,\$a2); print_r(\$result); ?></pre>	Array ([b] => green [d] => blue)
array_fill()	Fills an array with values	<pre>\$a1=array_fill(3,4,"blue"); print_r(\$a1);</pre>	Array ([3] => blue [4] => blue [5] => blue [6] => blue)
array_flip()	Flips/Exchanges all keys with their associated values in an array	<pre>\$a1=array("a"=>"red","b"=>"green", "b"=>"blue","d"=>"yellow"); \$result=array_flip(\$a1); print_r(\$result);</pre>	Array ([red] => a [green] => b [blue] => c [yellow] => d)
array_intersect()	Compare arrays, and returns the matches (compare	<pre>\$a1=array("a"=>"red","b"=>"green", "b"=>"blue","d"=>"yellow"); \$a2=array("e"=>"red","f"=>"green", "b"=>"blue");</pre>	Array ([a] => red [b] => green [c] => blue)

Unit 1: Core PHP Programming

	values only)	<pre>\$result=array_intersect(\$a1,\$a2); print_r(\$result);</pre>	
array_key_exists()	Checks if the specified key exists in the array	<pre>\$a=array("Volvo"=>"XC90","BMW"=>"X5"); if (array_key_exists("Volvo",\$a)) { echo "Key exists!"; } else { echo "Key does not exist!"; }</pre>	Key exists!
array_merge()	Merges one or more arrays into one array	<pre>\$a1=array("red","green"); \$a2=array("blue","yellow"); print_r(array_merge(\$a1,\$a2));</pre>	Array ([0] => red [1] => green [2] => blue [3] => yellow)
array_multisort()	Sorts multiple or multi-dimensional arrays	<pre>\$a=array("Dog","Cat","Horse","Bear","Zebra"); array_multisort(\$a); print_r(\$a);</pre>	Array ([0] => Bear [1] => Cat [2] => Dog [3] => Horse [4] => Zebra)
array_pad()	Inserts a specified number of items, with a specified value, to an array	<pre>\$a=array("red","green"); print_r(array_pad(\$a,5,"blue"));</pre>	Array ([0] => red [1] => green [2] => blue [3] => blue [4] => blue)
array_pop()	Deletes the last element of an array	<pre>\$a=array("red","green","blue"); array_pop(\$a); print_r(\$a);</pre>	Array ([0] => red [1] => green)
array_push()	Inserts one or more elements to the end of an array	<pre>\$a=array("red","green"); array_push(\$a,"blue","yellow"); print_r(\$a);</pre>	Array ([0] => red [1] => green [2] => blue [3] => yellow)
array_replace()	Replaces the values of the first array	<pre>\$a1=array("red","green"); \$a2=array("blue","yellow"); print_r(array_replace(\$a1,\$a2));</pre>	Array ([0] => blue [1] => yellow)

Unit 1: Core PHP Programming

	with the values from following arrays		
array_reverse()	Returns an array in the reverse order	<pre>\$a=array("a"=>"Volvo","b"=>"BMW", "c"=>"Toyota"); print_r(array_reverse(\$a));</pre>	Array ([c] => Toyota [b] => BMW [a] => Volvo)
array_search()	Searches an array for a given value and returns the key	<pre>\$a=array("a"=>"red","b"=>"green","c" =>"blue"); echo array_search("red",\$a);</pre>	a
array_slice()	Returns selected parts of an array	<pre>\$a=array("red","green","blue","yellow", "brown"); print_r(array_slice(\$a,2));</pre>	Array ([0] => blue [1] => yellow [2] => brown)
array_sum()	Returns the sum of the values in an array	<pre>\$a=array(5,15,25); echo array_sum(\$a);</pre>	45
array_unique()	Removes duplicate values from an array	<pre>\$a=array("a"=>"red","b"=>"green","c" =>"red"); print_r(array_unique(\$a));</pre>	Array ([a] => red [b] => green)
arsort()	Sorts an associative array in descending order, according to the value	<pre>\$age=array("Peter"=>"35","Ben"=>"37", "Joe"=>"43"); arsort(\$age); foreach(\$age as \$x=>\$x_value) { echo "Key=" . \$x . ", Value=" . \$x_value; echo "
"; }</pre>	Key=Joe, Value=43 Key=Ben, Value=37 Key=Peter, Value=35
asort()	Sorts an associative array in ascending	<pre>\$age=array("Peter"=>"35","Ben"=>"37", "Joe"=>"43"); asort(\$age);</pre>	Key=Peter, Value=35 Key=Ben, Value=37 Key=Joe, Value=43

Unit 1: Core PHP Programming

	order, according to the value	<pre>foreach(\$age as \$x=>\$x_value) { echo "Key=" . \$x . ", Value=" . \$x_value; echo "
"; }</pre>	
compact()	Create array containing variables and their values	<pre>\$firstname = "Peter"; \$lastname = "Griffin"; \$age = "41"; \$result = compact("firstname", "lastname", "a ge"); print_r(\$result);</pre>	Array ([firstname] => Peter [lastname] => Griffin [age] => 41)
count()	Returns the number of elements in an array	<pre>\$cars=array("Volvo","BMW","Toyota "); echo count(\$cars);</pre>	3
in_array()	Checks if a specified value exists in an array	<pre>\$people = array("Peter", "Joe", "Glenn", "Cleveland"); if (in_array("Glenn", \$people)) { echo "Match found"; } else { echo "Match not found"; }</pre>	Match found
krsort()	Sorts an associative array in descending order, according to the key	<pre>\$age=array("Peter"=>"35","Ben"=>"3 7","Joe"=>"43"); krsort(\$age); foreach(\$age as \$x=>\$x_value) { echo "Key=" . \$x . ", Value=" . \$x_value;</pre>	Key=Peter, Value=35 Key=Joe, Value=43 Key=Ben, Value=37

Unit 1: Core PHP Programming

		<pre>echo "
"; }</pre>	
ksort()	Sorts an associative array in ascending order, according to the key	<pre>\$age=array("Peter"=>"35","Ben"=>"37","Joe"=>"43"); ksort(\$age); foreach(\$age as \$x=>\$x_value) { echo "Key=" . \$x . ", Value=" . \$x_value; echo "
"; }</pre>	Key=Ben, Value=37 Key=Joe, Value=43 Key=Peter, Value=35
rsort()	Sorts an indexed array in descending order	<pre>\$cars=array("Volvo","BMW","Toyota"); rsort(\$cars);</pre>	Volvo Toyota BMW
sort()	Sorts an indexed array in ascending order	<pre>\$cars=array("Volvo","BMW","Toyota"); sort(\$cars);</pre>	BMW Toyota Volvo

Variable Handling Functions

Function	Description
boolval()	Returns the boolean value of a variable
empty()	Checks whether a variable is empty
is_array()	Checks whether a variable is an array
is_bool()	Checks whether a variable is a Boolean
is_float()/is_double()	Checks whether a variable is of type float
is_int()/is_integer()	Checks whether a variable is of type integer
is_null()	Checks whether a variable is NULL
is_numeric()	Checks whether a variable is a number or a numeric string
is_string()	Checks whether a variable is of type string
isset()	Checks whether a variable is set (declared and not NULL)

Unit 1: Core PHP Programming

print_r()	Prints the information about a variable in a human-readable way
unset()	Unsets a variable
var_dump()	Dumps information about one or more variables

Miscellaneous functions:

Function	Description	Example
Define	<code>define(name,value,case_insensitive)</code> A constant's value cannot be changed after it is set Constant names do not need a leading dollar sign (\$) Constants can be accessed regardless of scope Constant values can only be strings and numbers	<code>define("pi",3.14);</code> <code>echo pi;</code> Output:3.14
Exit()	<code>exit(message)</code> prints a message and terminates the current script	<code>\$x = 1;</code> <code>exit (\$x);</code>
Die()	<code>die(message)</code> Print a message and terminate the current script Note: <code>exit()</code> is used to stop the execution of the program, and <code>die()</code> is used to throw an exception and stop the execution.	<code>mysql_connect("hostname","mysqlusername","")</code> or <code>die('We are aware of the problem and working on it');</code>
Header	<code>header(header, replace, http_response_code)</code> Sends a raw HTTP header to a client	<code>header("Expires: Mon, 26 Jul 1997 05:00:00 GMT");</code> <code>header("Cache-Control: no-cache");</code>

User-defined Functions

We can declare and call user-defined functions easily

Syntax

```
function functionname(arguments){  
    //code to be executed  
    Return;  
}
```

Example:

```
<?php  
function sayHello(){  
    echo "Hello PHP Function";  
}  
sayHello();//calling function  
?>
```

Function Arguments

We can pass the information in PHP function through arguments which is separated by comma. PHP supports **Call by Value** (default), **Call by Reference**, **Default argument values** and **Variable-length argument list**.

Call by value:

```
<?php  
  
function funSum($n1,$n2)  
{  
    $ans=$n1+$n2;  
    echo $ans;  
}  
$x=10;  
$y=20;  
funSum($x,$y);  
?>
```

Call by reference:

Unit 1: Core PHP Programming

By default, value passed to the function is call by value. To pass value as a reference, you need to use ampersand (&) symbol before the argument name.

```
<?php
function adder(&$str2)
{
    $str2 .= 'Call By Reference';
}
$str = 'Hello ';
adder($str);
echo $str;
?>
```

Default Argument Value

We can specify a default argument value in function. While calling PHP function if you don't specify any argument, it will take the default argument

```
<?php
function sayHello($name="Sonoo"){
echo "Hello $name<br/>";
}
sayHello("Rajesh");
sayHello();//passing no value
sayHello("John");
?>
```

Returning Value

```
<?php
function cube($n){
return $n*$n*$n;
}
echo "Cube of 3 is: ".cube(3);
?>
```

we have passed two parameters **\$x** and **\$y** inside two functions **add()** and **sub()**.

```
<?php
//add() function with two parameter
function add($x,$y)
{
    $sum=$x+$y;
    echo "Sum = $sum <br><br>";
}
```

Unit 1: Core PHP Programming

```
}  
//sub() function with two parameter  
function sub($x,$y)  
{  
    $sub=$x-$y;  
    echo "Diff = $sub <br><br>";  
}  
//call function, get two argument through input box and click on add or sub button  
if(isset($_POST['add']))  
{  
    //call add() function  
    add($_POST['first'],$_POST['second']);  
}  
if(isset($_POST['sub']))  
{  
    //call add() function  
    sub($_POST['first'],$_POST['second']);  
}  
?>  
<form method="post">  
Enter first number: <input type="number" name="first"/><br><br>  
Enter second number: <input type="number" name="second"/><br><br>  
<input type="submit" name="add" value="ADDITION"/>  
<input type="submit" name="sub" value="SUBTRACTION"/>  
</form>
```

GET,POST and REQUEST methods

There are two ways the browser client can send information to the web server.

- The GET Method
- The POST Method

Before the browser sends the information, it encodes it using a scheme called URL encoding. In this scheme, name/value pairs are joined with equal signs and different pairs are separated by the ampersand.

name1=value1&name2=value2&name3=value3

Spaces are removed and replaced with the + character and any other nonalphanumeric characters are replaced with a hexadecimal values. After the information is encoded it is sent to the server.

The GET Method

The GET method sends the encoded user information appended to the page request. The page and the encoded information are separated by the ? character.

`http://www.test.com/index.htm?name1=value1&name2=value2`

- The GET method produces a long string that appears in your server logs, in the browser's Location: box.
- The GET method is restricted to send upto 1024 characters only.
- Never use GET method if you have password or other sensitive information to be sent to the server.
- GET can't be used to send binary data, like images or word documents, to the server.
- The data sent by GET method can be accessed using QUERY_STRING environment variable.
- The PHP provides `$_GET` associative array to access all the sent information using GET method.

Try out following example by putting the source code in test.php script.

```
<?php
if( $_GET["name"] || $_GET["age"] ) {
    echo "Welcome ". $_GET['name']. "<br />";
    echo "You are ". $_GET['age']. " years old.";

    exit();
}
?>
<html>
<body>

<form action = "<?php $_PHP_SELF ?>" method = "GET">
    Name: <input type = "text" name = "name" />
    Age: <input type = "text" name = "age" />
    <input type = "submit" />
</form>

</body>
</html>
```

The POST Method

The POST method transfers information via HTTP headers. The information is encoded as described in case of GET method and put into a header called QUERY_STRING.

- The POST method does not have any restriction on data size to be sent.
- The POST method can be used to send ASCII as well as binary data.
- The data sent by POST method goes through HTTP header so security depends on HTTP protocol. By using Secure HTTP you can make sure that your information is secure.
- The PHP provides **\$_POST** associative array to access all the sent information using POST method.

Try out following example by putting the source code in test.php script.

```
<?php
if( $_POST["name"] || $_POST["age"] ) {
    if (preg_match("/^[A-Za-z'-]$/",$_POST['name'])) {
        die ("invalid name and name should be alpha");
    }
    echo "Welcome ". $_POST['name']. "<br />";
    echo "You are ". $_POST['age']. " years old.";

    exit();
}
?>
<html>
<body>

<form action = "<?php $_PHP_SELF ?>" method = "POST">
    Name: <input type = "text" name = "name" />
    Age: <input type = "text" name = "age" />
    <input type = "submit" />
</form>

</body>
</html>
```

The \$_REQUEST variable

The PHP \$_REQUEST variable contains the contents of both \$_GET, \$_POST, and \$_COOKIE. We will discuss \$_COOKIE variable when we will explain about cookies.

Unit 1: Core PHP Programming

The PHP `$_REQUEST` variable can be used to get the result from form data sent with both the GET and POST methods.

Try out following example by putting the source code in test.php script.

```
<?php
if( $_REQUEST["name"] || $_REQUEST["age"] ) {
    echo "Welcome ". $_REQUEST['name']. "<br />";
    echo "You are ". $_REQUEST['age']. " years old.";
    exit();
}
?>
<html>
<body>

<form action = "<?php $_PHP_SELF ?>" method = "POST">
    Name: <input type = "text" name = "name" />
    Age: <input type = "text" name = "age" />
    <input type = "submit" />
</form>

</body>
</html>
```

Here `$_PHP_SELF` variable contains the name of self script in which it is being called.

Input Validation and Sanitization:

Input validation and sanitization are **essential techniques** in PHP to ensure that data received from users is safe and usable. These steps protect your application from attacks like XSS, SQL injection, and malformed input.

It can be performed with the use of `filter_var()` function.

`filter_var()` is a **built-in PHP function** used to **validate** and **sanitize** data (mostly user input). It applies a **filter** to a variable and returns the filtered result.

Syntax:

`filter_var(variable, filter, options)`

- variable: The value to be filtered.

Unit 1: Core PHP Programming

- **filter:** The ID of the filter to apply (like `FILTER_VALIDATE_EMAIL`, `FILTER_SANITIZE_STRING`, etc.).
- **options:** Optional. An associative array of options or flags to customize behavior.

Why Use `filter_var()`?

- Prevents security vulnerabilities (like XSS, injection attacks).
- Ensures data integrity (correct format and type).
- Simplifies input handling with built-in filters.

Validation Example:

Check if an email is valid:

```
$email = "test@example.com";
```

```
if (filter_var($email, FILTER_VALIDATE_EMAIL)) {  
    echo "Valid email";  
} else {  
    echo "Invalid email";  
}
```

Sanitization Example:

Remove illegal characters from an email:

```
$email = "test@@example.com";
```

```
$clean_email = filter_var($email, FILTER_SANITIZE_EMAIL);
```

```
echo $clean_email; // Outputs: test@@example.com (removes invalid parts)
```

Common Filters:

Filter Type	Filter Name	Description
Validation	<code>FILTER_VALIDATE_INT</code>	Validates integer
	<code>FILTER_VALIDATE_EMAIL</code>	Validates email format
	<code>FILTER_VALIDATE_URL</code>	Validates URL format
Sanitization	<code>FILTER_SANITIZE_STRING</code>	Removes HTML tags
	<code>FILTER_SANITIZE_EMAIL</code>	Removes illegal characters in email
	<code>FILTER_SANITIZE_URL</code>	Removes illegal URL characters

Note: As of PHP 8.1, `FILTER_SANITIZE_STRING` is deprecated. Use other methods like `strip_tags()` and `htmlspecialchars()` for strings.

Unit 1: Core PHP Programming

1. Input Validation

Input **validation** checks whether the user input meets expected rules (like format, type, length, etc.).

Example: Validating Email Address

```
$email = $_POST['email'];
```

```
if (filter_var($email, FILTER_VALIDATE_EMAIL)) {  
    echo "Valid email: " . $email;  
} else {  
    echo "Invalid email format.";  
}
```

Example: Validating Integer

```
$age = $_POST['age'];
```

```
if (filter_var($age, FILTER_VALIDATE_INT)) {  
    echo "Valid age: " . $age;  
} else {  
    echo "Age must be an integer.";  
}
```

2. Input Sanitization

Sanitization **cleans** input by removing or escaping unwanted characters to make it safe for further processing.

Example: Sanitizing Email Input

```
php
```

```
CopyEdit
```

```
$email = $_POST['email'];
```

```
$clean_email = filter_var($email, FILTER_SANITIZE_EMAIL);
```

```
echo "Sanitized email: " . $clean_email;
```

Example: Sanitizing String to Remove HTML Tags

```
php
```

```
CopyEdit
```

```
$name = $_POST['name'];
```

```
$clean_name = strip_tags($name);
```

```
echo "Sanitized name: " . $clean_name;
```